

JavaScript 1.2

Values, Types, Equality & Coercion

Print-first interview refresher · model values first, then follow the operator

Core question:

What value is here?

Which conversion and equality rule applies?

1 · Values, identity & mutation

Type belongs to the current value. Reassignment belongs to a binding. Mutation belongs to an object identity.

Primitive values · immutable

undefined · null · boolean · number
bigint · string · symbol

Object values · identity

objects · arrays · functions · dates · maps...
properties may mutate; equality compares identity

caller binding
reference value A

parameter gets a copy
reference value A

shared object identity A
mutation visible · reassignment local

2 · Type checks

```
typeof undefined    "undefined"
typeof null          "object" *
typeof NaN           "number"
typeof 1n            "bigint"
typeof Symbol()      "symbol"
typeof []            "object"
typeof function(){}  "function"
* historical behavior; check null explicitly
```

Identifier trap

```
typeof neverDeclared "undefined"
typeof later // TDZ      ReferenceError
typeof does not bypass the TDZ.
```

BigInt integers only; division truncates
`1n + 1 → TypeError`

3 · Absence & defaults

Falsy

false · 0 · -0 · 0n · NaN
"" · null · undefined

Nullish null · undefined

`value || fallback`
fallback for any falsy value

`value ?? fallback`
fallback only for nullish value

Keep valid 0, "" and false → use ??

Optional chaining

user?.profile?.name
Continuous chain stops on nullish base.
`(obj?.a).b // chain broken`

4 · Equality semantics

Rule	NaN	±0	Used by
===	no	same	indexOf
Object.is	yes	diff	SameValue
SameValueZero	yes	same	Set / Map

Production default

=== / !== same type + identity/value

Abstract equality ==

May coerce types. Trace its algorithm;
do not memorize isolated puzzles.

```
value == null
true only for null / undefined*
* ordinary values; document team policy
```

`{ } === { } → false`

5 · Coercion: normalize, then operate

raw input → validate grammar → convert explicitly → domain type

Explicit conversion

`Boolean(x) · Number(x) · String(x)`
Then validate finite / range / safe integer.

Binary + has two branches

ToPrimitive both operands → any string?

```
"5" + 2 → "52"          "5" - 2 → 3
1n + 1 → TypeError
```

ToPrimitive(object, hint)

- Symbol.toPrimitive(hint), if present
- string hint: toString → valueOf
- number/default: valueOf → toString

Must return a primitive; Date default is string-like.

6 · Numeric edges & production safety

NaN NaN !== NaN `Number.isNaN(x)`
Global isNaN(x) coerces first.

Infinity `Number.isFinite(x)`
Not NaN does not imply finite or valid.

Signed zero -0 === 0 `Object.is(-0, 0) → false`

Floating point

`0.1 + 0.2 !== 0.3`

Measurements: abs + relative domain tolerance

`diff ≤ max(absTol, relTol × max(|a|, |b|))`

`Number.EPSILON` is spacing near 1, not a policy.

Money · fixed scale

integer minor units + safe-range checks
+ explicit rounding points and policy

Output-prediction method

1 values/types → 2 short-circuit/unary → 3 ToPrimitive → 4 operator conversion → 5 equality → 6 value + type + error
Production: normalize at boundaries · keep stable types · make ownership explicit · prefer === · use domain comparisons.